

Section 4.4 — Modern Data Platforms: Warehouses, Lakes & Lakehouses

Enhanced Study Notes | 5 Files in Series (4.1–4.5)

Study Directive (5 files): This is file 4 of 5 in the Section 4 database series. This section is **architecture-heavy and comparison-heavy**. Give extra weight to before/after platform diagrams, comparisons between Redshift/BigQuery/Snowflake/ ClickHouse, ETL vs ELT trade-offs, and the lakehouse pattern as an evolution of the data lake.

Executive Overview

Modern analytics stacks have evolved from single data warehouses to multi-layer platforms. The data warehouse era (1990s–2010) gave way to the data lake era (2010–2018), which is now converging into the lakehouse pattern (2018–present). Understanding each layer’s purpose, limitations, and the tools that implement them is essential for designing data-intensive systems.

1. Data Warehouse Architecture

1.1 Why Columnar Storage Wins for Analytics

BEFORE — Row-oriented storage (OLTP):

Table row layout on disk:

```
[user_id=1 | name="Alice" | country="US" | age=30 | email="a@b.com" | ...]  
[user_id=2 | name="Bob" | country="UK" | age=25 | email="c@d.com" | ...]  
[user_id=3 | name="Carol" | country="US" | age=35 | email="e@f.com" | ...]
```

Query: `SELECT AVG(age) FROM users WHERE country = 'US';`

Must read: ALL columns of ALL rows → reads name, email, etc. (irrelevant)

At 1GB table: reads 1GB from disk.

AFTER — Column-oriented storage (OLAP):

Country column stored contiguously: [US, UK, US, AU, US, AU, UK, ...]

Age column stored contiguously: [30, 25, 35, 28, 41, 33, 29, ...]

Query: `SELECT AVG(age) FROM users WHERE country = 'US';`
Step 1: Read country column (200MB) → bitmap of US rows [1, 0, 1, 0, 1, ...]
Step 2: Read age column (200MB), apply bitmap → sum only US ages
Total: 400MB read (vs 1GB) = 2.5x speedup from column pruning alone.

With compression (run-length encoding on country column):
[US × 30,000 | UK × 20,000 | AU × 15,000 | ...]
Uncompressed: 3 bytes × 100,000 = 300KB
RLE compressed: 3 entries = 24 bytes → 12,500x compression for this column

With SIMD vectorized execution:
Load 8 age values into a 256-bit AVX register, compare all 8 to threshold
Process 8 values per CPU instruction (vs 1 for scalar code)
At 3GHz: 3,000,000,000 / 8 = 375M values/second per core
32 cores: 12 billion values/second → scan 1B rows in < 1 second

With predicate pushdown to storage (Parquet/ORC):
Row group statistics: min_age=18, max_age=65 per 128MB chunk
Query: `WHERE age > 100` → skip entire row group (min/max pruning)
Eliminates I/O for chunks that cannot possibly match the predicate

1.2 MPP (Massively Parallel Processing) Architecture

Query execution flow on Snowflake/Redshift:

SQL query arrives at Leader/Coordinator node:

```
SELECT region, SUM(sales) FROM orders
WHERE year = 2024
GROUP BY region;
```

Leader node:

1. Parse and optimize query plan
2. Split work into N slices (one per compute node)
3. Broadcast `WHERE year = 2024` filter to all nodes
4. Each node scans its local data slice in parallel

Compute Node 1 (owns 1/N of data):

Scan local orders partition → filter year=2024 → local SUM by region
Send partial result to Leader

Compute Node 2... N: (same in parallel)

Leader collects partial results from all nodes:

Merge partial SUMs by region → final answer

Linear speedup: 2 nodes → 2x faster (ideal). Real: ~1.8x (coordination overhead).

At 100 nodes vs 1 node: 100GB scan → 1GB per node in parallel → 100x speedup.

Node-local data distribution (critical for avoiding skew):

Round-robin: each row goes to next node in rotation (uniform, bad for joins)

Hash distribution: DISTKEY → rows with same key on same node (enables co-located joins)

Replicated: small dimension tables copied to all nodes (avoids broadcast at query time)

Example – Redshift distribution:

```
CREATE TABLE orders DISTKEY(customer_id) SORTKEY(order_date);  
CREATE TABLE customers DISTKEY(customer_id);  
JOIN orders ON customers.id = orders.customer_id → co-located → no data  
movement
```

1.3 Warehouse Comparison Matrix

Feature	Redshift	BigQuery	Snowflake	ClickHouse
Compute model	Dedicated cluster	Serverless slots	Multi-cluster	Dedicated
Storage	Managed (RA3: S3)	Colossus (Google)	S3/Azure/GCS	Local NVMe
Concurrency	~50 queries	Near-unlimited	Excellent	Fair
Cold start	None (always on)	~2s per query	Seconds	None
Pricing model	Per node/hour	Per TB scanned	Per credit	Per node/hour
Best for	AWS-native, predictable load	Ad-hoc, variable load	Multi-cloud, elastic	Real-time analytics
Max query speed	Fast	Very fast	Very fast	Extremely fast
Surprise cost risk	Low	HIGH (scan cost)	Medium	Low

Choosing a warehouse – decision guide:

Already on AWS, predictable analytics load?

→ Redshift (RA3 nodes: compute-storage separation, familiar SQL)

Variable load, bursty analytics, GCP or cloud-agnostic?

👉 BigQuery (serverless: pay per query, no cluster management)

Watch out: full table scan = expensive. Always use partitioned tables.

Multi-cloud, multiple teams sharing compute, elastic scaling?

→ Snowflake (virtual warehouses: each team gets its own compute, shared storage)

Sub-second analytics on real-time data (<100ms queries)?

→ ClickHouse (columnar, vectorized, MergeTree engine, extreme read

performance)

Example: Cloudflare uses ClickHouse for real-time web analytics at petabyte scale.

Cost example – BigQuery gotcha:

SELECT * FROM events; ← scans entire 10TB table → \$50 per query!

Fix: Always use WHERE partition_date = '2024-01-15' → scans 1 day = \$0.25

Rule: NEVER SELECT * from BigQuery without a partition filter.

2. Data Lake Architecture

2.1 Core Data Lake Design

BEFORE (data warehouse only):

Problem 1: Warehouse schema is rigid → new data requires schema change + ETL rewrite

Problem 2: Raw data discarded after transformation → can't reprocess with new logic

Problem 3: Only structured data → can't store logs, images, ML training sets

Problem 4: Expensive storage → can't keep 10 years of data affordably

AFTER (data lake on object storage):

Landing zone (raw, immutable):

s3://company-lake/

raw/

events/year=2024/month=01/day=15/hour=14/

events_20240115_140000.json.gz ← original, never modified

logs/year=2024/...

clickstream/year=2024/...

Processed zone (cleaned, enriched):

s3://company-lake/

processed/

events/year=2024/month=01/day=15/

events_clean.parquet ← Parquet: columnar, compressed

Curated zone (aggregated, business-ready):

s3://company-lake/

curated/

daily_active_users/date=2024-01-15/

dau.parquet

Schema-on-read principle:

Data stored without schema enforcement.

Schema applied at query time by the analytics engine (Spark, Athena, Presto).

New field in event JSON? → available immediately, no migration.

Downside: schema drift is silent → field renamed → old queries break quietly.

2.2 File Format Comparison

Format	Layout	Compression	Predicate Pushdown	Best For
JSON/CSV	Row	Low	No	Raw landing, human readable
Avro	Row	Good	No	Kafka messages, schema registry
Parquet	Columnar	Excellent	Yes	Analytics, Spark default
ORC	Columnar	Excellent	Yes	Hive workloads
Delta/Iceberg	Columnar (Parquet) + txn log	Excellent	Yes	Lakehouse ACID

Parquet internal structure:

File → Row Groups (128MB each) → Columns → Pages (1MB each)

Row Group metadata:

For each column: min, max, null_count, distinct_count

Used for: min/max pruning (skip row group if value out of range)

Column chunk compression:

Dictionary encoding: first appearance stored, subsequent = dictionary index

Example: country column with 200 countries in 10M rows

Without dict: "United States" × 3M = 42MB

With dict: entry 0 = "United States", 3M × 2bytes = 6MB (7x savings)

On-disk size comparison for 1TB raw JSON logs:

JSON (gzip): 200GB

Parquet (snappy): 60GB (3x smaller than gzip JSON)

Parquet (zstd): 40GB (5x smaller)

Query speed: Parquet 50x faster than JSON (columnar + predicate pushdown)

3. Lakehouse Architecture — ACID on Object Storage

3.1 The Problem with Plain Data Lakes

Data lake without ACID (plain S3 Parquet files):

Problem 1 – Failed partial writes are not rolled back:

Spark job writes 100 Parquet files to S3.

```
Job fails after 50 files.
```

```
Directory contains: 50 good files + 0 bad files (job failed before writing bad
ones).
```

But: which 50 completed? What about partial data?

No transaction log → no way to know what's valid.

Readers may query corrupt/incomplete data.

Problem 2 – No schema enforcement:

Engineer A writes events with field: {"amount": 100.0} (float)

Engineer B writes events with field: {"amount": "100"} (string – bug)

No constraint catches this. Analytics silently computes wrong sums.

Problem 3 – No concurrent writes:

Two Spark jobs write to same Parquet directory simultaneously.

Files overwrite each other silently. Data loss.

Problem 4 – No efficient deletes:

"Right to be forgotten" (GDPR): `DELETE FROM events WHERE user_id = 42`

Parquet files are immutable. Must rewrite entire partition. Very slow.

3.2 Delta Lake — ACID for Object Storage

Delta Lake adds a transaction log (`_delta_log/`) on top of Parquet files:

```
s3://my-table/_delta_log/
  00000000000000000000000000000000.json ← transaction 0: initial table creation
  00000000000000000000000000000001.json ← transaction 1: added 3 files
  00000000000000000000000000000002.json ← transaction 2: deleted 1 file
  00000000000000000000000000000003.json ← transaction 3: schema change
part-00000-abc.parquet ← data file
part-00001-def.parquet ← data file
part-00002-ghi.parquet ← data file (deleted in txn 2, but file stays
until VACUUM)
```

Transaction log entry (JSON):

```
{
  "add": {
    "path": "part-00003-jkl.parquet",
    "size": 1234567,
    "stats": {"numRecords": 50000, "minValues": {"ts": "2024-01-15"}, ...}
  }
}
```

```
}
```

ACID guarantees:

Atomicity: write all files → then write one `_delta_log` entry atomically
S3 atomic PUT: either the log entry exists or it doesn't
Failure mid-write: log entry absent → files are "orphaned" (VACUUM cleans up)

Isolation: Readers read a consistent snapshot (specific log version)
Writer conflict detection: optimistic concurrency on log file names
If two writers try to write log entry 00005: only one succeeds (S3 atomic PUT)
Loser must retry.

Durability: Log entries in S3 are durable (11 nines)
Consistency: Schema validation on write (enforced by Delta library)

Time travel:

```
SELECT * FROM events VERSION AS OF 42;      ← see table as it was at version 42
SELECT * FROM events TIMESTAMP AS OF '2024-01-15 10:30:00';
```

Implementation: read `_delta_log` up to the requested version, reconstruct file list.

Use case: audit trail, reproduce ML training datasets, debug production incidents.

VACUUM (remove old files):

VACUUM events RETAIN 168 HOURS; ← keep 7 days of history, delete older files
After VACUUM: time travel only works for the retention window.

3.3 Lakehouse Format Comparison

Feature	Delta Lake	Apache Iceberg	Apache Hudi
ACID	Full (OCC)	Full (OCC)	Full (OCC)
Time travel	✓ Version + timestamp	✓ Snapshot-based	✓
Schema evolution	✓ Add/drop/rename cols	✓ Excellent	✓
Streaming writes	Structured Streaming	Flink, Kafka	Spark, Flink
Primary backer	Databricks	Netflix/Apple	Uber
Query engine	Spark, Trino, DuckDB	Spark, Trino, Athena	Spark, Presto
Upsert/delete	MERGE INTO	MERGE	Optimized (native upsert)

Feature	Delta Lake	Apache Iceberg	Apache Hudi
Best for	Databricks users, Spark	Multi-engine, cloud-native	Incremental ingestion, CDC

4. ETL vs ELT — When to Transform Where

4.1 Architecture Comparison

TRADITIONAL ETL (Extract → Transform → Load):

```
Source DB → [ETL Server] → Data Warehouse
                ↑
            Transform happens in intermediate compute server
            Data warehouse receives clean, structured data only
```

Problems:

- Transform server = bottleneck (limited compute)
- Raw data not preserved (can't re-transform with new logic)
- Schema changes require ETL rewrite
- Tools: Informatica, Talend, SSIS (complex, expensive)

MODERN ELT (Extract → Load → Transform):

```
Source DB → Object Storage (raw) → Data Warehouse (transform with SQL)
                                      ↑
                                Warehouse compute scales
                                Raw data preserved in lake
                                dbt transforms inside warehouse (SQL)
```

ELT tools:

- Fivetran / Airbyte: managed Extract + Load (100+ connectors)
- dbt: Transform inside warehouse using SQL + version control
- Airflow / Prefect: Orchestration (schedule, dependencies, retry)

dbt transformation example:

```
-- models/marts/daily_revenue.sql
WITH orders AS (
  SELECT DATE(created_at) as date, SUM(total) as revenue
  FROM {{ ref('stg_orders') }}      ← references staging model
  WHERE status = 'completed'
  GROUP BY 1
),
targets AS (
  SELECT date, target_revenue
  FROM {{ ref('revenue_targets') }}
)
SELECT o.date, o.revenue, t.target_revenue,
       o.revenue / t.target_revenue as attainment
```



```
FROM orders o LEFT JOIN targets t USING (date)
```

dbt run → compiles to SQL → executes in warehouse → creates table/view

dbt test → runs assertions: no nulls, unique IDs, referential integrity

dbt docs → generates data lineage graph automatically

4.2 Orchestration Tools Comparison

Tool	Reliability	Exactly-Once	UI Quality	Best For
Airflow	Good	Best-effort	Excellent	Complex DAGs, large ecosystem
Prefect	Good	With config	Good	Python-native, easier to use
Dagster	Good	Good	Excellent	Data-aware orchestration, assets
Temporal	Excellent	Guaranteed	Fair	Durable workflows, microservices
dbt Cloud	Good	Good	Good	SQL transformations only

When to use each:

Airflow:   Established team, complex cross-system DAGs, large plugin ecosystem
Steep learning curve, brittle dynamic DAGs, local dev complex

Dagster:   "Software-defined assets" model, great lineage visibility
 Better testing story than Airflow
 Newer, smaller community

Temporal:   Long-running business workflows (not just data pipelines)
 Guaranteed exactly-once execution with retry built-in
 Handles failures, timeouts, signals (pause/resume)
 Not a data pipeline tool natively – use for orchestrating services

Recommendation for pure data engineering: Dagster or Airflow + dbt

Recommendation for microservice workflows: Temporal

5. Interview Use Cases & Design Problems

Problem 1 — Design a Near-Real-Time Analytics Pipeline (1TB/day)

Problem statement: Design a pipeline to process 1TB of raw application logs per day and make them queryable within 1 hour. Optimize for cost (budget: < \$500/month) and freshness.

Solution:

Cost constraint analysis:

1TB/day raw → ~300GB/day after compression (Parquet/zstd: 3x compression)

S3 storage: 300GB/day × 30 days = 9TB × \$0.023/GB = \$207/month

Compute: need ~2 hours/day of Spark compute

Architecture — Micro-batch lakehouse:

INGEST (continuous):

Apps → Kinesis/Kafka → S3 landing zone (raw JSON)

Partition: /raw/logs/year=YYYY/month=MM/day=DD/hour=HH/

Cost: Kinesis ~\$30/month for this volume

PROCESS (hourly, triggered):

Airflow DAG: trigger at :05 past each hour

Spark job (EMR Serverless or Databricks Serverless):

- Read last hour's raw JSON from S3
- Parse, validate, deduplicate (using event_id)
- Write to Delta Lake: /processed/logs/ partitioned by date+hour

Duration: ~20 minutes for 40GB/hour of logs

Cost: EMR Serverless ~\$0.0815/vCPU-hour + \$0.0114/GB-hour

Per run: 4 vCPUs × 20 min = ~\$0.11/run × 24 = \$2.64/day = \$79/month

SERVE (query):

Athena: serverless SQL on Delta Lake in S3

Query: SELECT ... FROM logs WHERE date = '2024-01-15' AND hour = 14;

Cost: \$5/TB scanned × 0.04TB (Parquet efficient) = \$0.20/query

Budget queries: 1000/month = \$200/month

Total cost: \$207 (storage) + \$79 (processing) + \$200 (queries) = \$486/month 

Freshness: raw data arrives → hourly job starts at :05 → done by :25 → queryable by :30

SLA: 1-hour freshness → easily met (30 min in practice)

Streaming alternative for < 5 minute freshness:

Kafka → Flink → Hudi (incremental upserts) → always-on compute

Cost: ~\$3,000/month (Flink cluster always running)

Conclusion: micro-batch is 6x cheaper for a 1-hour SLA.

Always use micro-batch when SLA allows it.

Problem 2 — Migrating from a Monolithic Data Warehouse to a Lakehouse

Problem statement: Your company has a 50TB Redshift warehouse. It's slow for ad-hoc queries, can't handle ML workloads, and costs \$80K/year. How do you migrate to a lakehouse without disrupting existing BI reports?

Solution:

Phase 1 — Parallel-build (Month 1-2): Zero disruption

Set up Delta Lake on S3: `s3://company-lakehouse/`

Start CDC (Change Data Capture) from Redshift source systems → S3 landing

Tools: AWS DMS or Debezium → streams all changes to S3 raw layer

Run dbt models on Spark to recreate existing Redshift tables in Delta Lake

Validate: compare query results between Redshift and lakehouse (5% sample)

Phase 2 — Read migration (Month 3): Low risk

Migrate low-stakes consumers first:

Data science notebooks → point to lakehouse (Spark/Databricks)

Ad-hoc exploration → Athena on Delta Lake

Keep BI reports (Tableau/Looker) pointing to Redshift

Monitor: query latency, result accuracy, user complaints

Phase 3 — BI migration (Month 4-5): Careful

Migrate dashboards one by one to query lakehouse (Trino/Athena)

Use Redshift as fallback for 30 days

Validate each dashboard result matches within 1% of Redshift

Phase 4 — Decommission Redshift (Month 6):

Scale Redshift cluster down → savings start

Keep Redshift in "snapshot" mode for 6 months (rollback option)

Delete after 6 months → full savings realized

Cost comparison:

Before: Redshift `ra3.4xlarge` × 4 nodes = \$80K/year

After: S3 storage ($50\text{TB} \times \$0.023$) = \$1,380/month = \$16.5K/year)

+ Athena queries ($\$5/\text{TB scanned} \times \text{managed usage}$) = ~\$10K/year)

+ dbt Cloud (\$2K/year)

Total after: ~\$28K/year → 65% cost reduction

Risks and mitigations:

Query performance regression: Redshift optimized for specific query patterns.

Mitigation: Benchmark top 20 queries before migration. Optimize Delta Lake partition strategy to match.

Operational complexity increase: Lakehouse has more moving parts.

Mitigation: Managed Databricks or Starburst reduces operational burden.

Problem 3 — GDPR “Right to Be Forgotten” in a Data Lake

Problem statement: Your data lake stores 5 years of event logs in Parquet on S3. A user requests deletion of all their data (GDPR Article 17). How do you implement this efficiently without rewriting 5 years of Parquet files?

Solution:

Approach A — Rewrite affected partitions (simple but slow):

Find all partitions containing `user_id = 42`:

```
SELECT DISTINCT year, month, day FROM events WHERE user_id = 42;
```

For each partition: rewrite Parquet file excluding user 42.

At 5 years $\times$ 365 days = 1825 partitions, each 1GB:

Rewrite 1825 files $\times$ 1 minute each = 30 hours. Unacceptable.

Approach B — Deletion log (fast, Delta Lake native):

```
CREATE TABLE deletion_requests (  
  user_id BIGINT,  
  requested_at TIMESTAMP,  
  executed BOOLEAN DEFAULT false  
);
```

```
INSERT INTO deletion_requests VALUES (42, NOW(), false);
```

All queries on events JOIN to filter deleted users:

```
SELECT e.* FROM events e  
LEFT JOIN deletion_requests d ON e.user_id = d.user_id  
WHERE d.user_id IS NULL;
```

Views for all analytics:

```
CREATE VIEW events_gdpr_safe AS SELECT ... (with deletion filter)
```

Background compaction job (weekly):

Rewrites old partitions to physically remove deleted user data.

Only partitions older than 30 days (cooling period).

Time: spread across week = $1825 / 7 = 260$ partitions/day $\times$ 1 min = 4 hours/day.

Approach C — Pseudonymization + key deletion (cryptographic erasure):

At ingest: `user_id` is encrypted: `encrypted_user_id = AES(user_id, user_key)`

`user_key` stored separately in KMS (AWS KMS or HashiCorp Vault) keyed by `user_id`.

Events stored with: `encrypted_user_id` (not raw `user_id`)

On deletion request:

```
DELETE key from KMS for user 42.
```

All `encrypted_user_id` values for user 42 are now undecryptable (garbage).

Effectively "deleted" cryptographically without touching Parquet files!

Compliance: most GDPR implementations accept this as sufficient erasure.

Recommended: Approach C for new systems (design for it upfront).

Approach B for existing systems (minimal disruption).
Approach A only for small datasets or compliance edge cases.